

Build a complete, production-style, cloud-based web application called “**Smart OR Scheduler – Operating Room Scheduling & Optimization System**”.

1. PROJECT PURPOSE

Create an intelligent hospital Operating Room (OR) scheduling system that automatically assigns surgeries to operating rooms and time slots while respecting:

- Surgery duration
- Surgery priority
- Emergency/urgency level
- Deadlines
- Operating-room working hours
- Room availability
- Room type and equipment requirements
- No overlapping surgeries
- Available hospital resources

The system must be **algorithm-centric**, not just a CRUD application. It must clearly show the scheduling decisions, algorithm steps, constraints, objective value, and reason for every assignment.

The application should be responsive and work smoothly on desktop, tablet, and mobile.

2. TECHNOLOGY STACK

Use:

- **Frontend:** HTML5, CSS3, JavaScript
- **Backend:** Python 3 + Flask
- **Database:** PostgreSQL
- **Algorithms:** Python
- **Priority Queue:** Python heap-based priority queue
- **Scheduling Strategy:** Greedy Algorithm
- **Job Scheduling:** Job Sequencing with deadlines and duration-aware slot allocation
- **REST API:** Flask REST endpoints
- **Charts:** JavaScript chart library or native visualizations
- **Cloud:** Deploy the complete application as a cloud-hosted web application
- **Authentication:** Secure login system with session/token-based authentication
- **Environment Variables:** Store database credentials and secrets securely

Use clean modular architecture so the frontend, backend, database, and scheduling engine are separated.

3. MAIN USERS

Support these roles:

Admin

Can:

- Manage users
- Add/edit/delete surgeries
- Add/edit/delete operating rooms
- Configure working hours
- Configure scheduling rules
- Run scheduling
- View analytics
- View decision logs

Scheduler / Hospital Staff

Can:

- Enter surgeries
- Manage room availability
- Run the scheduling algorithm
- Review assignments
- Modify scheduling inputs
- View conflicts and unscheduled surgeries

Viewer

Can:

- View schedules
- View room utilization
- View algorithm decisions
- View analytics

4. AUTHENTICATION

Create a professional login page.

Include:

- Email
- Password
- Login
- Logout
- Role-based access
- Password validation
- Protected pages
- Secure session handling

After login, redirect users to the dashboard.

5. DATABASE DESIGN

Use PostgreSQL.

Create these tables:

users

Fields:

- id
- name
- email
- password_hash
- role
- created_at

surgeries

Fields:

- id
- patient_name
- patient_id
- surgery_name
- surgery_type
- duration_minutes
- priority_level
- urgency_level
- requested_start
- deadline
- required_room_type
- required_equipment
- surgeon_name
- status
- created_at

operating_rooms

Fields:

- id
- room_name
- room_type
- equipment

- opening_time
- closing_time
- availability_status

room_unavailability

Fields:

- id
- room_id
- start_time
- end_time
- reason

schedules

Fields:

- id
- surgery_id
- room_id
- start_time
- end_time
- scheduling_score
- status
- created_at

scheduling_logs

Fields:

- id
- surgery_id
- selected_priority
- candidate_rooms
- candidate_slots
- selected_room
- selected_slot
- decision
- reason
- score
- created_at

scheduling_runs

Fields:

- id

- run_name
- total_surgeries
- scheduled_count
- unscheduled_count
- objective_value
- room_utilization
- average_waiting_time
- deadline_compliance
- created_at

6. INPUT DATA

Provide forms to enter all required scheduling information.

Surgery Form

Fields:

- Patient Name
- Patient ID
- Surgery Name
- Surgery Type
- Duration
- Priority
- Urgency
- Deadline
- Surgeon
- Required Room Type
- Required Equipment

Priority levels:

- Emergency
- Critical
- High
- Medium
- Low

Use numerical priority scores internally, for example:

Emergency = 5 Critical = 4 High = 3 Medium = 2 Low = 1

Make the priority rule visible to the user.

Operating Room Form

Fields:

- Room Name
- Room Type
- Opening Time
- Closing Time
- Equipment
- Availability

Allow staff to define temporary unavailable periods.

7. CORE ALGORITHM ENGINE

Implement three core algorithmic components.

A. PRIORITY QUEUE

Use a heap-based priority queue.

The queue should repeatedly select the surgery with the highest scheduling priority.

Create an explicit priority score.

Example:

Priority Score = Medical Priority

- Urgency Score
- Deadline Urgency
- Waiting-Time Factor

The exact weights must be configurable.

Display the priority calculation for each surgery.

Example:

Surgery S101

Medical Priority = 5 Urgency = 4 Deadline Urgency = 3

Total Score = 12

Show this calculation in the UI.

8. B. GREEDY SCHEDULING

Use a greedy strategy.

For each selected surgery:

1. Retrieve available operating rooms.
2. Remove incompatible rooms.
3. Find all feasible start times.
4. Check whether the complete duration fits.
5. Check deadline.
6. Check conflicts.
7. Calculate the score for each feasible assignment.
8. Select the best feasible local assignment.

Use this greedy rule:

“Select the highest-priority surgery first and assign it to the earliest feasible time slot in the compatible room while minimizing unnecessary idle time and respecting all constraints.”

Make this rule visible in the application.

9. C. JOB SEQUENCING

Implement duration-aware job sequencing.

Each surgery is treated as a job with:

- Duration
- Deadline
- Priority
- Required resources

The scheduler must reserve **continuous time** for the entire surgery.

For example:

A surgery requiring 120 minutes cannot be split into:

08:00–09:00 and 11:00–12:00.

It must receive one continuous slot:

08:00–10:00.

The system should search available continuous intervals.

10. HARD CONSTRAINTS

The scheduling engine **MUST** enforce these constraints:

No overlapping surgeries

A room cannot have two surgeries at the same time.

Room availability

A surgery cannot run outside room working hours.

Duration

The assigned interval must exactly accommodate the surgery duration.

Deadline

The surgery should finish no later than its specified deadline when feasible.

Room compatibility

A surgery can only use a room that satisfies its required type/equipment.

Temporary room closure

Unavailable periods must be blocked.

Resource conflicts

Prevent scheduling conflicts for resources defined as exclusive.

11. SOFT OBJECTIVES

When several feasible choices exist, optimize the schedule using configurable objectives.

Possible objectives:

- Maximize high-priority surgeries scheduled
- Minimize patient waiting time
- Minimize deadline violations
- Minimize room idle time
- Maximize operating-room utilization
- Minimize unnecessary gaps between surgeries

Create a configurable objective-weight section.

Example:

Priority Weight = 40% Deadline Weight = 25% Room Utilization = 20% Waiting Time = 15%

Allow administrators to modify these weights.

12. EXPLAINABLE DECISION ENGINE

This is a critical requirement.

Do NOT only display the final schedule.

For every scheduling decision, show:

- Selected surgery
- Priority score
- Candidate rooms
- Candidate time slots
- Rejected rooms
- Rejected slots
- Constraint causing rejection
- Selected room
- Selected start/end time
- Scheduling score
- Human-readable explanation

Example:

“Surgery S101 was selected because it has the highest priority score of 12. OR-1 and OR-2 were evaluated. OR-2 was unavailable during the required interval. OR-1 had a continuous 120-minute slot from 08:00 to 10:00, which satisfies the room requirements and deadline. Therefore S101 was assigned to OR-1.”

Store this explanation in `scheduling_logs`.

13. SCHEDULING WORKFLOW

Implement this complete workflow:

Input Surgeries ↓ Input Operating Rooms ↓ Load Constraints ↓ Calculate Priority Scores ↓ Build Priority Queue ↓ Select Highest-Priority Surgery ↓ Find Compatible Rooms ↓ Find Feasible Continuous Time Slots ↓ Check Constraints ↓ Calculate Candidate Scores ↓ Greedy Selection ↓ Assign Room + Time ↓ Update Room Availability ↓ Store Decision Log ↓ Process Next Surgery ↓ Repeat Until Queue Empty ↓ Generate Final Schedule ↓ Calculate Objective Value ↓ Show Analytics

14. DASHBOARD

Create a modern dashboard.

Display:

- Total Surgeries
- Scheduled Surgeries
- Unscheduled Surgeries
- Emergency Surgeries
- Operating Rooms
- Room Utilization
- Average Waiting Time
- Deadline Compliance
- Objective Score

Use attractive cards and charts.

15. LIVE SCHEDULE VIEW

Create a visual timeline/calendar.

Example:

OPERATING ROOM SCHEDULE			
OR-1			
08:00	——	10:00	S101
10:00	——	11:30	S105
11:30	——	13:00	S110
13:00	——	14:00	AVAILABLE
OR-2			
08:00	——	09:30	S102
09:30	——	12:30	S103
12:30	——	14:00	AVAILABLE

Use different visual states for:

- Emergency
- Critical
- High
- Medium
- Low
- Available

- Unavailable
- Conflict

16. ALGORITHM VISUALIZER

Create a dedicated page named:

“Algorithm Decision Center”

Show the scheduling process step by step.

Example:

```
STEP 1
Priority Queue

S101 → Score 12
S103 → Score 10
S102 → Score 8
S105 → Score 7

↓

STEP 2
Select S101

↓

STEP 3
Check Rooms

OR-1 ✓
OR-2 ✓
OR-3 ✗

↓

STEP 4
Feasible Slots

OR-1 → 08:00–10:00 ✓
OR-1 → 13:00–15:00 ✗ deadline
OR-2 → 08:30–10:30 ✓

↓
```

STEP 5
Greedy Selection

Selected:
OR-1
08:00–10:00

↓

STEP 6
Update Schedule

Animate or progressively reveal these steps.

17. UNSCHEDULED SURGERIES

Create a dedicated section.

For every unscheduled surgery show:

- Surgery ID
- Priority
- Duration
- Deadline
- Required room
- Failure reason
- Conflicting resources
- Suggested next available slot

Example:

“S109 could not be scheduled because no compatible room had a continuous 180-minute availability before the deadline.”

18. ANALYTICS

Create a professional analytics page.

Include:

Room Utilization

OR-1 = 88%

OR-2 = 82%

OR-3 = 91%

Priority Distribution

Show number of:

- Emergency
- Critical
- High
- Medium
- Low

Scheduling Performance

Display:

- Scheduling success rate
- Deadline compliance
- Average waiting time
- Total idle time
- Total operating time
- Total unscheduled duration

Objective

Show:

Final Objective Value

and explain how it was calculated.

19. SIMULATION MODE

Add a "Run Scheduler" button.

When clicked:

1. Load all pending surgeries.
2. Build the priority queue.
3. Execute the scheduling engine.
4. Show algorithm progress.
5. Save assignments.

6. Generate logs.
7. Calculate metrics.
8. Display final schedule.

Also provide:

- Reset Schedule
- Run Again
- Compare Runs

20. SCHEDULE COMPARISON

Allow users to compare two scheduling runs.

Display:

- Objective value
- Scheduled surgeries
- Waiting time
- Room utilization
- Deadline compliance
- Unscheduled cases

Show which metrics changed between runs without hiding the underlying numbers.

21. API DESIGN

Create REST APIs such as:

POST `/api/auth/login`

GET `/api/surgeries`

POST `/api/surgeries`

PUT `/api/surgeries/{id}`

DELETE `/api/surgeries/{id}`

GET `/api/rooms`

POST `/api/rooms`

PUT `/api/rooms/{id}`

POST `/api/schedule/run`

GET `/api/schedule`

GET `/api/schedule/{id}`

GET `/api/schedule/logs`

GET `/api/analytics`

GET `/api/runs`

GET `/api/runs/{id}`

22. BACKEND STRUCTURE

Use a clean structure similar to:

```
backend/  
|  
├─ app.py  
├─ config.py  
├─ models/  
├─ routes/  
├─ services/  
├─ algorithms/  
|   ├─ priority_queue.py  
|   ├─ greedy_scheduler.py  
|   ├─ job_sequencing.py  
|   └─ constraint_checker.py  
├─ database/  
├─ utils/  
└─ tests/
```

Keep algorithm logic separate from Flask routes.

23. FRONTEND PAGES

Create:

1. Login
2. Dashboard
3. Surgery Management
4. Operating Room Management
5. Schedule
6. Algorithm Decision Center
7. Unscheduled Surgeries

8. Analytics
9. Scheduling Run History
10. Settings
11. User Management

24. DESIGN REQUIREMENTS

Use a modern hospital-tech dashboard design.

The UI should be:

- Professional
- Clean
- Responsive
- Modern
- Easy to understand
- Suitable for a college project demonstration and hackathon presentation

Use:

- Sidebar navigation
- Top navigation bar
- Dashboard cards
- Tables
- Timeline views
- Charts
- Modal forms
- Status badges
- Search and filter
- Sorting
- Pagination where needed

Use a consistent design system.

25. DEMO DATA

Automatically provide realistic demo data so the application works immediately after deployment.

Create:

- At least 10 surgeries
- At least 3 operating rooms
- Different durations
- Different priorities
- Different deadlines
- Different room requirements
- At least one unavailable room period

- At least one intentionally unschedulable surgery

Include a **“Load Demo Data”** button.

26. VALIDATION

Add frontend and backend validation.

Examples:

- Duration must be greater than zero.
- Deadline cannot be before requested time.
- Room closing time must be after opening time.
- Required room type must be valid.
- Priority must be one of the configured values.
- Surgery cannot have invalid duration.
- Prevent duplicate room assignments.

Display user-friendly error messages.

27. TESTING

Create automated tests for:

- Priority queue ordering
- Deadline handling
- Duration handling
- Room compatibility
- Overlap detection
- Room availability
- Greedy selection
- Job sequencing
- Unscheduled surgery detection
- Objective calculation

Include sample test cases.

28. SECURITY

Implement:

- Password hashing
- Authentication
- Authorization
- Input validation
- SQL injection protection
- Secure API handling

- Environment variables
- Error handling without exposing secrets

Do not hard-code database passwords or API keys.

29. CLOUD DEPLOYMENT

Prepare the application for cloud deployment.

Include:

- Production Flask configuration
- PostgreSQL connection through environment variables
- Database migration/setup
- Production WSGI server
- Health check endpoint
- Logging
- Error handling
- Deployment documentation

Environment variables should include:

```
DATABASE_URL
SECRET_KEY
FLASK_ENV
```

Create a README containing:

- Project overview
- Architecture
- Algorithms
- Database schema
- Installation
- Environment variables
- Local development
- Database setup
- Cloud deployment
- API documentation
- Testing instructions

30. FINAL RESULT PAGE

After running the algorithm, show:

Final Schedule

Surgery	Room	Start	End	Priority	Status
---------	------	-------	-----	----------	--------

Algorithm Summary

Show:

- Number of priority-queue selections
- Number of candidate assignments evaluated
- Number of rejected assignments
- Number of successful assignments
- Number of unscheduled surgeries

Objective Summary

Show:

- Objective value
- Room utilization
- Average waiting time
- Deadline compliance
- Total idle time

Explanation

For each surgery, show a readable scheduling explanation.

31. IMPORTANT ALGORITHM REQUIREMENT

The application must NOT fake algorithm execution using static data.

The schedule displayed in the UI must actually be generated by the Python scheduling engine.

The Priority Queue, Greedy Algorithm, Job Sequencing, constraint checking, scoring, and decision logging must be implemented as real executable code.

Every result shown on the dashboard must come from the actual database and scheduling engine.

32. ERROR HANDLING

Handle:

- Database connection errors
- Invalid input
- Scheduling conflicts

- No available room
- No feasible time slot
- Deadline violations
- API failures
- Authentication failures

Show clear messages rather than technical stack traces to normal users.

33. PROJECT DEMONSTRATION MODE

Add a “Demo Mode” that lets a user click:

Generate Demo Schedule

Then show the complete scheduling process:

```
Input Data
↓
Priority Calculation
↓
Priority Queue
↓
Surgery Selection
↓
Room Filtering
↓
Slot Search
↓
Constraint Validation
↓
Greedy Choice
↓
Assignment
↓
Decision Explanation
↓
Final Schedule
↓
Analytics
```

Make this visually impressive for a college project or hackathon demonstration.

34. FINAL EXPECTATION

Generate the complete working full-stack cloud application.

Do not create only a UI mockup.

Implement:

- Working frontend
- Working Flask backend
- Working PostgreSQL database
- Working Priority Queue
- Working Job Sequencing
- Working Greedy scheduler
- Working constraint engine
- Working APIs
- Working authentication
- Working analytics
- Working schedule timeline
- Working decision logs
- Working demo data
- Working tests
- Cloud deployment configuration

The final application should feel like a real hospital scheduling prototype and clearly demonstrate how algorithms are used to make scheduling decisions.